GUIDED LAB A · INFRASTRUCTURE FOR BIG DATA

Infrastructure as Code: The Cluster as a Vagrantfile

Build the cluster from code, work the Vagrant lifecycle, and grow it by one line

Student details	
Name	
Date	

Overview

> **Scenario:** In earlier labs you built a 3-node VirtualBox cluster by hand, one VM at a time; one mistyped IP was enough to break it. Today you define the same cluster as code, practise getting a broken node back, and grow the cluster by editing one line. Lab B continues from here in your second block.

You work individually on your lab PC. **Suggested time: about 55 minutes** of your first 1-hour block.

By the end of this lab you will be able to

- Build a multi-node cluster with `vagrant up` and check it from inside each VM
- Explain which line of a Vagrantfile replaces each manual build step
- Get a broken node back three ways: snapshot restore, re-provisioning and rebuilding
- Add a node by changing code, and re-provision the nodes that need to know about it
- Change a VM setting in code and explain the drift between code and running machines

Lab settings

Use this table whenever the lab tells you to look up a value.

Setting	Value
Lab folder	**iac-vagrant-lab\labA-cluster**
Box (base image)	`ubuntu/jammy64` (Ubuntu 22.04), pre-loaded on the lab PC
Cluster network	NAT Network **VagrantNet**, 10.20.20.0/24, DHCP off (created by the Vagrantfile)
Nodes	node01 10.20.20.11, node02 10.20.20.12, node03 10.20.20.13 (added in section 3)
Each node	1 GB RAM, 1 CPU. NIC1 = NAT for `vagrant ssh`; NIC2 (`enp0s8`) = VagrantNet
Inside each VM	user **student**, password `student` (lab only)

> **Watch out:** VagrantNet is separate from your **BigDataNet** (10.10.10.0/24). Nothing in this lab touches your manually built VMs or the network you will use in the practical exam.

Before you begin

1. Shut down your manually built VMs. This lab needs about 3 GB of free memory.
2. Open a terminal (PowerShell on Windows) and run `vagrant --version` and `VBoxManage --version`.
3. Run `vagrant box list` and check that `ubuntu/jammy64` is listed. If it is not, ask your tutor; do not download it over the lab network.

1 Bring up a 2-node cluster

> **Why this matters:** One command replaces every step of your manual build. The boot takes about 5 minutes, so you read the code while you wait and match it to what you did by hand.

1. Go to the lab folder and start the build:

```
cd iac-vagrant-lab/labA-cluster
vagrant up
```

2. While it runs, open the **Vagrantfile** in an editor. For each manual step below, write the line number(s) that replace it.

Manual step	Vagrantfile line(s)
Create the NAT Network, DHCP off	
Create an Ubuntu 22.04 VM	
Set RAM and CPUs	
Attach the NIC to the NAT Network	
Set a static IP in netplan	
Create the student user	
Edit /etc/hosts on every node	
Repeat all of the above for each VM	

3. When the build finishes, check it:

```
vagrant status
vagrant ssh node01
ip -4 addr show enp0s8
cat /etc/hosts
ping -c 3 node02
exit
```

What you found	Result
Both nodes show as running? (Y/N)	
IP address on node01's enp0s8	
node01 can ping node02 by name? (Y/N)	
Time from `vagrant up` to finished (minutes)	

> **Discussion:** `vagrant ssh` worked before any static IP existed. Which network card does it use, and why does the cluster need a second one?

Notes, observations, screenshots

2 Work the Vagrant lifecycle

Why this matters: In the manual build, a broken VM meant redoing steps or hoping you had a snapshot. With code you have three ways back: restore a snapshot, re-run the provisioning, or rebuild the machine. Each one suits a different kind of failure.

1. Power a node off and on again. Note that provisioning does not run a second time:

```
vagrant halt node02
vagrant status
vagrant up node02
```

2. Take a snapshot of node01, then break its name resolution by deleting the cluster block from /etc/hosts:

```
vagrant snapshot save node01 clean
vagrant ssh node01 -c "sudo sed -i '/# BEGIN cluster/,/# END cluster/d' /etc/hosts"
vagrant ssh node01 -c "ping -c 2 node02"
```

3. Recover with the snapshot, and check the ping again:

```
vagrant snapshot restore node01 clean
vagrant ssh node01 -c "ping -c 2 node02"
```

4. Break node01 the same way again (repeat the `sed` command from step 2). This time recover with `vagrant provision node01`, and check the ping.
5. Throw node02 away completely and rebuild it. Time the rebuild:

```
vagrant destroy -f node02
vagrant up node02
```

What you did	Result
State of node02 after `vagrant halt` (from `vagrant status`)	
Ping by name after breaking /etc/hosts (works / fails)	
Recovered by snapshot restore? (Y/N)	
Recovered by `vagrant provision`? (Y/N)	
Time to destroy and rebuild node02 (minutes)	

Discussion: Snapshot restore, re-provision, rebuild: when would you choose each one? Which of the three would still work if node01's virtual disk was deleted?

Notes, observations, screenshots

3 Add a third node

Why this matters: In the manual build, a new node meant cloning a VM and fixing its hostname, netplan and every /etc/hosts file by hand. Here it is one line of code, plus re-running the provisioning on the nodes that need to know about it.

1. Before you edit anything, write down in the notes box what you expect to change.
2. Add node03 to the **NODES** list in the Vagrantfile and save it:

```
NODES = {
  "node01" => "10.20.20.11",
  "node02" => "10.20.20.12",
  "node03" => "10.20.20.13",
}
```

3. Build only the new node, then test in both directions:

```
vagrant up node03
vagrant ssh node03 -c "ping -c 2 node01"
vagrant ssh node01 -c "ping -c 2 node03"
```

4. One of those pings fails. Look at `/etc/hosts` on node01 and explain why in the table below.
5. Re-run provisioning on the older nodes, then repeat the failing ping:

```
vagrant provision node01 node02
vagrant ssh node01 -c "ping -c 2 node03"
```

What you did	Result
node03 → node01 by name? (Y/N)	
node01 → node03 before re-provisioning? (Y/N)	
Why did that ping fail?	
node01 → node03 after re-provisioning? (Y/N)	
Lines of code you changed to add the node	

Discussion: Provisioning has now run several times on node01, yet /etc/hosts has no duplicate entries. Which line of the provisioning script makes that safe, and what is this property called?

Notes: what did adding a node take in the manual build, and how long?

4 Change a setting in code

Why this matters: Changing RAM or CPUs by hand means opening each VM's settings. In code it is one value for every node, but it only takes effect on the machines you apply it to.

1. Check how many CPUs node01 has: `vagrant ssh node01 -c "nproc"`
2. In the Vagrantfile, change `CPUS = 1` to `CPUS = 2` and save.
3. Apply the change to node01 only, then check again:

```
vagrant reload node01
vagrant ssh node01 -c "nproc"
vagrant ssh node02 -c "nproc"
```

What you found	Result
node01 CPUs before / after	
node02 CPUs now	
Does every running node match the Vagrantfile? (Y/N)	

Discussion: The Vagrantfile now says 2 CPUs, but node02 and node03 still have 1. What is this mismatch called, and which single command would bring every node back in line with the code?

Notes, observations, screenshots

5 Final check and save your work

> **Why this matters:** Lab B starts from the Vagrantfile you finish with today. Your VMs will be destroyed; your code is what you keep.

1. Copy your **Vagrantfile** somewhere you will have next week (OneDrive, USB key or your own Git repository).
2. Fill in the check table, then destroy the cluster:

```
vagrant status
vagrant destroy -f
vagrant status
```

Check	Expected	Actual
Nodes running before destroy	node01, node02, node03	
node01 → node03 by name	Works	
CPUs on node01	2	
Where you saved your Vagrantfile	Somewhere you can reach in Lab B	
VMs left after `vagrant destroy -f`	None (all not created)	

Lab complete: check before you leave

- Mapping table in section 1 filled in
- node01 recovered by snapshot and by re-provisioning; node02 rebuilt from scratch
- node03 added with one line and all three nodes reachable by name
- Vagrantfile saved for Lab B, and all lab VMs destroyed
- Mistakes and lessons log started (below)

> **Discussion:** Compare your section 1 build time with the time the manual build took you. Where did the manual build lose most of its time?

Mistakes and lessons log: what went wrong, how you found it, what you would do differently next time.

Next: Lab B rebuilds this cluster from your Vagrantfile, installs Nginx on every node as code, and has you repair a broken Vagrantfile.